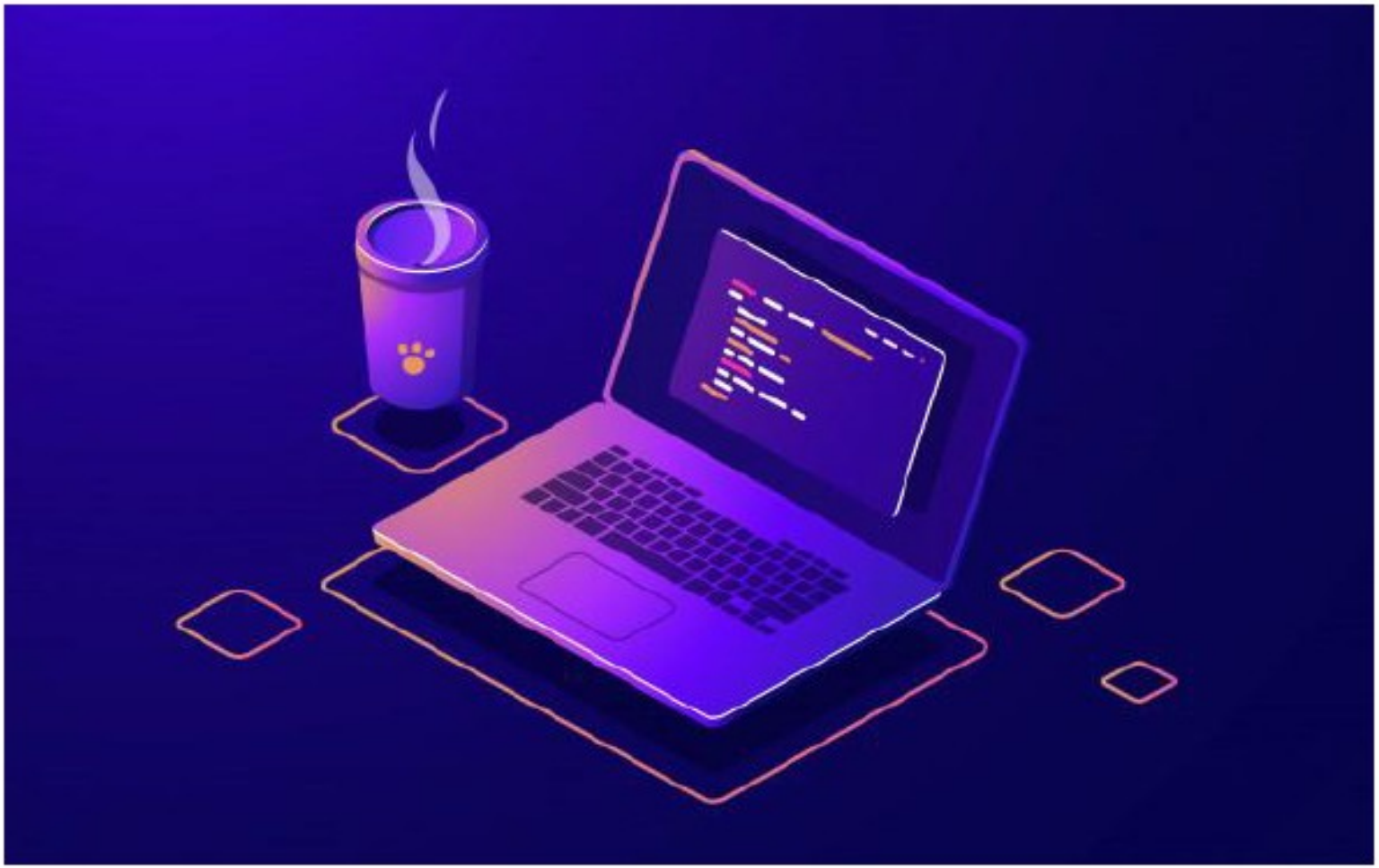
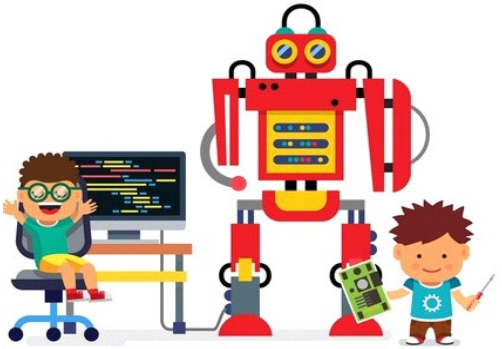


PROGRAMACIÓN

TEMA 5: PROGRAMACIÓN ORIENTADA A OBJETOS. UTILIZACIÓN AVANZADA DE CLASES

POO. UTILIZACIÓN AVANZADA DE CLASES

- CLASES ENVOLTORIO (WRAPPERS)
- TRABAJANDO CON FECHAS Y HORAS (LA CLASE DATE)
- CLASES Y MÉTODOS ABSTRACTOS Y FINALES.
- POLIMORFISMO.
- SOBREESCRITURA DE MÉTODOS.
- SOBRECARGA DE MÉTODOS (OVERLOADING).
- CONVERSIONES ENTRE OBJETOS (CASTING).
- ACCESO A MÉTODOS DE LA SUPERCLASE.
- CLASES ANIDADAS.



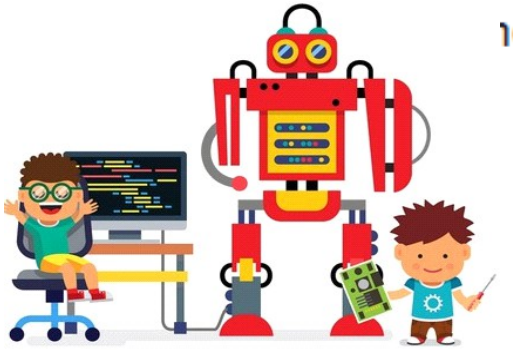
POO. UTILIZACIÓN AVANZADA DE CLASES

CLASES ENVOLTORIO (WRAPPERS)

La principal diferencia entre un tipo primitivo y un **wrapper** es que éste último es una clase. Cuando trabajamos con wrappers estamos trabajando con objetos mientras que cuando trabajamos con un tipo primitivo obviamente no.

A simple vista un wrapper siendo un objeto, puede aportar muchas ventajas pero, ¿qué problemas nos puede plantear utilizar objetos en vez de tipos primitivos?

El problema que nos podemos encontrar es que cuando le pasamos una variable a un método como argumento y esta es de un tipo primitivo se le pasa siempre por valor mientras que cuando pasamos un wrapper (objeto) se lo ıos pasando por referencia.



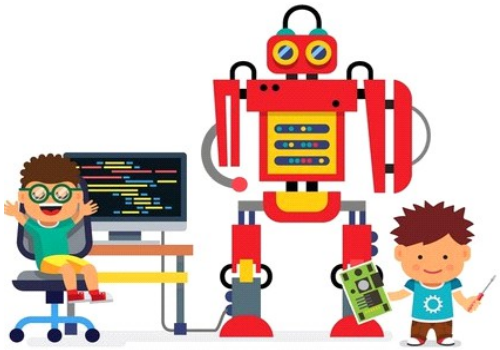
Echa un vistazo al apartado del tema 4 en el que se explica el paso de parámetros por valor y referencia.

POO. UTILIZACIÓN AVANZADA DE CLASES

CLASES ENVOLTORIO (WRAPPERS)

Una de las grandes ventajas que tienen estos wrapper es la facilidad de conversión entre tipos primitivos y cadenas de caracteres en ambos sentidos.

Tipo primitivo	Wrapper asociado
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean



POO. UTILIZACIÓN AVANZADA DE CLASES

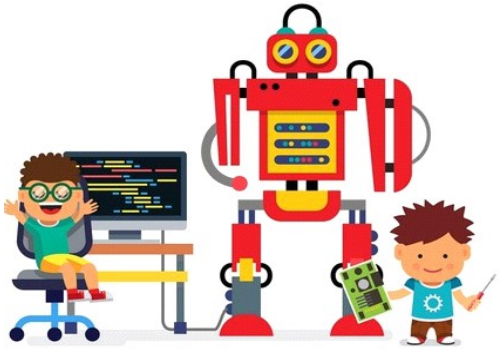
CLASES ENVOLTORIO (WRAPPERS)

El wrapper Integer

tiene dos constructores:

- Integer(int)
- Integer(String)

Método	Descripción
Integer(int) Integer(String)	Constructores.
byteValue() shortValue() intValue() longValue() doubleValue() floatValue()	Funciones de conversión con datos primitivos.
Integer decode(String) Integer parseInt(String) Integer parseInt(String,int) Integer valueOf(String) String toString()	Conversión a *String*.
String toBinaryString(int) String toHexString(int) String toOctalString(int)	Conversion a otros sistemas de numeración.
MAX_VALUE, MIN_VALUE, TYPE	Constantes.



a vez asignado un valor, no puede modificarse. Habría que crear otro Integer.

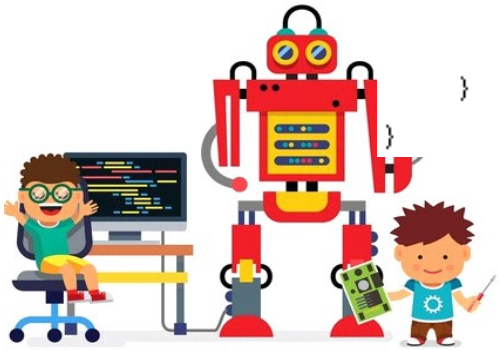
POO. UTILIZACIÓN AVANZADA DE CLASES

CLASES ENVOLTORIO (WRAPPERS)

El wrapper Integer

```
public class TestInteger {
    public static void main(String[] args){
        Integer i1 = new Integer(5);
        Integer i2 = new Integer("7");
        String s1 = i1.toString();
        System.out.println(s1);
        int i3 = Integer.parseInt("10",10);
        int i4 = Integer.parseInt("10",8);
        int i5 = Integer.parseInt("BABA",16);
        System.out.println("Muestra el 10 de base 10 en base 10: " + i3);
        System.out.println("Muestra el 10 de base 8 en base 10: " + i4);
        System.out.println("Muestra BABA de base 16 en base 10: " + i5);
        System.out.println("Muestra el 10 en base 8: "+Integer.toOctalString(i4));
        System.out.println("Muestra BABA en base hexadecimal: "+Integer.toHexString(i5));
        int i6 = Integer.valueOf("22").intValue();
        System.out.println("Pasa el String '22' a entero 22: "+i6);
    }
}
```

```
run:
5
Muestra el 10 de base 10 en base 10: 10
Muestra el 10 de base 8 en base 10: 8
Muestra BABA de base 16 en base 10: 47802
Muestra el 10 en base 8: 10
Muestra BABA en base hexadecimal: baba
Pasa el String '22' a entero 22: 22
BUILD SUCCESSFUL (total time: 0 seconds)
```



POO. UTILIZACIÓN AVANZADA DE CLASES

CLASE Date (TRABAJANDO CON FECHAS Y HORAS)

- La fecha y la hora se almacenan en un Long como **milisegundos desde 01/01/1970**.
- la clase **GregorianCalendar** deriva de la clase abstracta **Calendar**.

-Horas → 0-23

-Días → 1-31

-Meses → 0-11

-Años → 4 dígitos.

Tiene muchas variables enteras:

-DAY_OF_WEEK

-DAY_OF_MONTH

-YEAR, MONTH, HOUR, MINUTE, SECOND

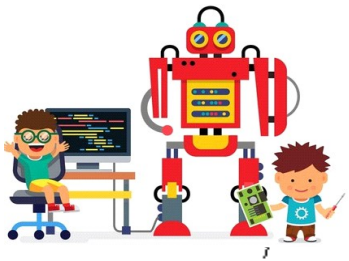
-WEEK_OF_MONTH.

-WEEK_OF_YEAR, ...

```java
import java.util.*;

public class fecha {
   public static void main(String [] args){
      Date d = new Date();
      GregorianCalendar c = new GregorianCalendar();
      c.setTime(d);
      System.out.println("Día del Mes: " + c.get(Calendar.DAY_OF_MONTH));
      System.out.println("Mes actual: " + c.get(Calendar.MONTH) + 1);
      System.out.println("Año actual: " + c.get(Calendar.YEAR));
      int hora = c.get(Calendar.HOUR_OF_DAY);
      int minutos = c.get(Calendar.MINUTE);
      int segundos = c.get(Calendar.SECOND);
      int milisegundos = c.get(Calendar.MILLISECOND);
      System.out.println("Hora actual -> " + hora + ":"
            + minutos + ":" + segundos + ":" + milisegundos);
```

```
run:
Día del Mes: 5
Mes actual: 01
Año actual: 2021
Hora actual -> 10:53:40:191
BUILD SUCCESSFUL (total time: 0 seconds)
```



POO. UTILIZACIÓN AVANZADA DE CLASES

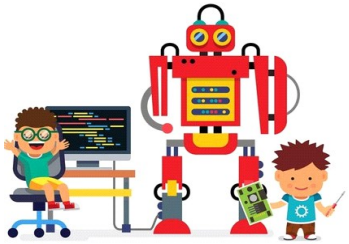
CLASE Date (TRABAJANDO CON FECHAS Y HORAS)

Java SimpleDateFormat - Java Date Format

Las clases de Java **SimpleDateFormat** y **DateFormat** se utilizan para formatear fechas. Se utiliza principalmente cuando necesitamos mostrar o utilizar la funcionalidad de fecha y hora de Java.

DateFormat

- DateFormat se utiliza para formatear una fecha String según la configuración regional específica que se proporciona como entrada.
- La configuración regional se utiliza para especificar la región y el idioma para que el código sea más local para el usuario.
- La forma de escribir la fecha es diferente en las distintas regiones del mundo. Por ejemplo, 31st Dec 2017 se escribirá en la India como 31-12-2017 pero lo mismo se escribirá en los Estados Unidos de América como 12-31-2017.
- DateFormat Las clases no están sincronizadas, se recomienda crear una instancia separada para cada hilo .



POO. UTILIZACIÓN AVANZADA DE CLASES

CLASE Date (TRABAJANDO CON FECHAS Y HORAS)

Java SimpleDateFormat - Java Date Format

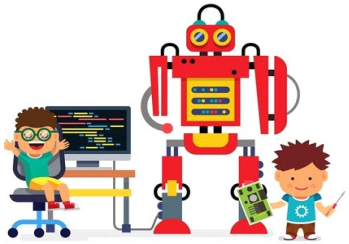
DateFormat

Se puede crear un objeto usando el método **getDateInstance()** y **getTimeInstance()** de la **DateFormat.**

```
Locale loc = new Locale("en", "US");
DateFormat dateFormat = DateFormat.getDateInstance(DateFormat.DEFAULT, loc);
```

El constructor **Locale(String, String)** está en desuso desde la versión 19.

```
Locale loc = new Locale.Builder().setLanguage("en").setRegion("US").build();
```



POO. UTILIZACIÓN AVANZADA DE CLASES

CLASE Date (TRABAJANDO CON FECHAS Y HORAS)

Java SimpleDateFormat - Java Date Format

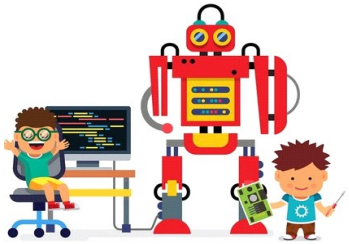
DateFormat

DateFormat tiene un método **format** que es responsable del formato.

```
Locale locale = new Locale("fr", "FR");
DateFormat dateFormat = DateFormat.getDateInstance(DateFormat.DEFAULT, locale);
String date = dateFormat.format(new Date());
System.out.print(date);
```

Este código formateará la fecha en el fridioma francés ( ) y en la FRregión de Francia ( ):

3 janv. 2018



POO. UTILIZACIÓN AVANZADA DE CLASES

CLASE Date (TRABAJANDO CON FECHAS Y HORAS)

Java SimpleDateFormat - Java Date Format

DateFormat

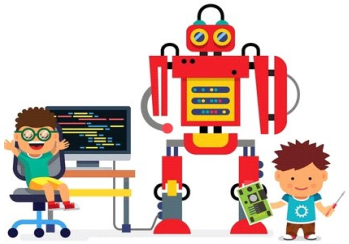
Para crear una instancia **DateFormat** estamos usando el método **getDateInstance()**.

Para realizar un formato de hora, necesitamos una instancia de hora. Usaremos **getTimeInstance()** un método para obtener una instancia de tiempo.

```
Locale locale = new Locale("fr", "FR");
DateFormat dateFormat = DateFormat.getTimeInstance(DateFormat.DEFAULT, locale);
String date = dateFormat.format(new Date());
System.out.print(date);
```

Este código formateará la hora en el idioma francés ( ) y en la región de Francia ( )

11:03:01



POO. UTILIZACIÓN AVANZADA DE CLASES

CLASE Date (TRABAJANDO CON FECHAS Y HORAS)

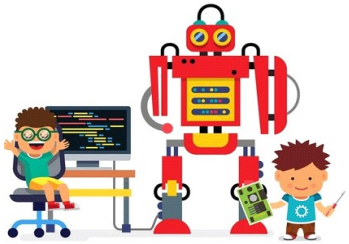
Java SimpleDateFormat - Java Date Format

SimpleDateFormat

SimpleDateFormat es parecido a **DateFormat**. La única diferencia importante entre ellos es que SimpleDateFormat se puede usar para formatear **(Dateconversión String)** y para analizar **(StringconversiónDate)** con soporte local, mientras que DateFormat no tienen soporte local.

SimpleDateFormat se puede crear usando el constructor SimpleDateFormat. El constructor es un **constructor parametrizado** y necesita un patrón de cadena como parámetro.

```
String pattern = "MM-dd-yyyy";
SimpleDateFormat simpleDateFormat = new SimpleDateFormat(pattern);
```



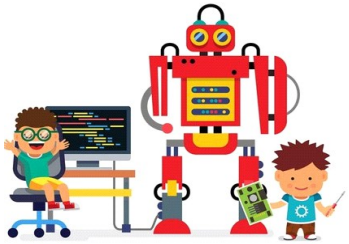
La Cadena **pattern** es el patrón que se utilizará para formatear una fecha y la salida se generará en ese patrón como "MM-dd-yyyy".

POO. UTILIZACIÓN AVANZADA DE CLASES

CLASE Date (TRABAJANDO CON FECHAS Y HORAS)

Java SimpleDateFormat - Java Date Format

G	designador de era	ANUNCIO
y	Año	2018 (aaaa), 18 (aa)
M	Mes en año	julio (MMMM), julio (MMM), 07 (MM)
w	Resultados en semana en año	dieciséis
W.	Resultados en semana en mes	3
D	Da el recuento de días en el año.	266
d	Día del mes	09 (día), 9(d)
F	Día de la semana en el mes.	4
E	Nombre del día de la semana.	martes, martes
u	Número de día de la semana donde 1 representa el lunes, 2 representa el martes y así sucesivamente	2
a	Marcador AM o PM	SOY
h	Hora del día (0-23)	12
k	Hora del día (1-24)	23
k	Hora en am/pm para formato de 12 horas (0-11)	0
h	Hora en am/pm para formato de 12 horas (1-12)	12
m	Minuto en la hora	59
s	Segundo en el minuto	35
S	Milisegundo en el minuto	978
z	Zona horaria	Hora estándar del Pacífico; hora del Pacífico; GMT-08:00
z	Desplazamiento de zona horaria en horas (patrón RFC)	-0800
X	Desplazamiento de zona horaria en formato ISO	-08; -0800; -08:00



POO. UTILIZACIÓN AVANZADA DE CLASES

CLASE Date (TRABAJANDO CON FECHAS Y HORAS)

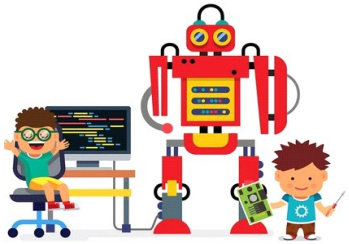
Java SimpleDateFormat - Java Date Format

Veamos ahora algunos ejemplos de diferentes formatos de fecha y hora.

Patrón	Resultado
MM/dd/yyyy	02/01/2018
dd-M-yyyy hh:mm:ss	02-1-2018 06:07:59
dd MMMM yyyy	02 enero 2018
dd MMMM yyyy zzzz	02 de enero de 2018, hora estándar de la India
E, dd MMM yyyy HH:mm:ss z	Martes, 02 de enero de 2018 18:07:59 IST

```
String pattern = "MM-dd-yyyy";
SimpleDateFormat simpleDateFormat = new SimpleDateFormat(pattern);
String date = simpleDateFormat.format(new Date());
System.out.println(date);
```

01-02-2018



POO. UTILIZACIÓN AVANZADA DE CLASES

CLASE Date (TRABAJANDO CON FECHAS Y HORAS)

Java SimpleDateFormat - Java Date Format

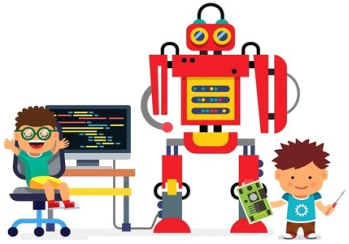
El tipo de carácter (y las mayúsculas) y la cantidad de caracteres contribuyen a la cadena. El mes se representaría con dos dígitos, el día con dos dígitos y el año con cuatro dígitos.

Este código mostraba la fecha del “2 de enero de 2018” como “ 01-02-2018”.

Para el tiempo de análisis, tenemos que cambiar el patrón mientras creamos SimpleDateFormatla instancia.

```
String pattern = "HH:mm:ss.SSSZ";
SimpleDateFormat simpleDateFormat = new SimpleDateFormat(pattern);
String date = simpleDateFormat.format(new Date());
System.out.println(date);
```

13:03:15.454+0530



Este patrón produce “horas”, “minutos”, “segundos”, “milisegundos” y “desplazamiento de zona horaria en horas (patrón RFC)”.

POO. UTILIZACIÓN AVANZADA DE CLASES

CLASE Date (TRABAJANDO CON FECHAS Y HORAS)

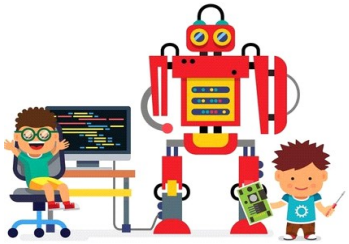
Java SimpleDateFormat - Java Date Format

parse()

El análisis es la conversión de String en una instancia java.util.Date. Podemos analizar una cadena en una instancia de fecha usando el método de la clase SimpleDateFormat **parse()**.

```
String pattern = "MM-dd-yyyy";
SimpleDateFormat simpleDateFormat = new SimpleDateFormat(pattern);
Date date = simpleDateFormat.parse("12-01-2018");
System.out.println(date);                    Sat Dec 01 00:00:00 IST 2018
```

```
String pattern = "HH:mm:ss";
SimpleDateFormat simpleDateFormat = new SimpleDateFormat(pattern);
Date date = simpleDateFormat.parse("22:00:03");
System.out.println(date);                    Thu Jan 01 22:00:03 IST 1970
```



POO. UTILIZACIÓN AVANZADA DE CLASES

CLASE Date (TRABAJANDO CON FECHAS Y HORAS)

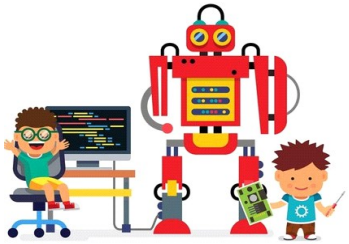
Java SimpleDateFormat - Java Date Format

Parse()

El análisis es la conversión de String en una instancia java.util.Date. Podemos analizar una cadena en una instancia de fecha usando el método de la clase SimpleDateFormat **parse()**.

```
String pattern = "EEEEE MMMMM yyyy HH:mm:ss.SSSZ";
SimpleDateFormat simDF = new SimpleDateFormat(pattern, new Locale("fr", "FR"));
Date date = simDF.format(new Date());
System.out.println(date);
```

mardi janvier 2018 14:51:02.354+0530



POO. UTILIZACIÓN AVANZADA DE CLASES

CLASES Y MÉTODOS ABSTRACTOS:

- Clases abstractas → Clases genéricas.
- No habrá objetos de estas clases.
- Se usan objetos de las subclases que heredan de ella.
- Se usan los atributos y métodos de la clase genérica en las subclases.
- Los métodos y atributos pueden ser abstract, o no.
- Los métodos abstract, no puede ser static. Las subclases deberán redefinirlos o declararlos abstract.

```
public abstract class vehiculo {
    private int peso;
    public void setPeso(int p) {peso=p;}
    public abstract int getVelocidadActual();
}
```

OBJETOS, CLASES Y MÉTODOS FINALES:

- Objeto final → impide que haya otro objeto con su referencia.
- Método final → Método que no puede sobreescribirse.

  public final void setcolor(String s) {color=s;}

  -El compilador puede colocar el bytecode en el sitio.

  -Ganancia de eficiencia.
- Clase final → No puede tener descendencia.

```
public class fecha {
    public static class cuadrado{
        int area;
        private cuadrado(int i) {
            area = i*i;
        }
    }

    public static void main(String [] args){
        final cuadrado c1;
        c1 = new cuadrado(5);
        cuadrado c2;
        c2 = new cuadrado(15);
        c1=c2;
    }
```

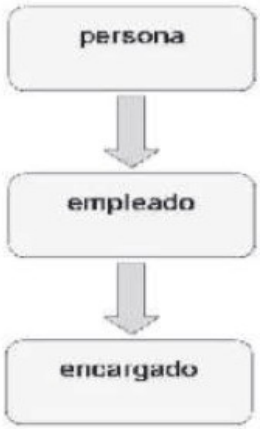
```
final cuadrado c1;
variable c1 might already have been assigned
----
(Alt-Enter shows hints)
c1=c2;
```

POO. UTILIZACIÓN AVANZADA DE CLASES

POLIMORFISMO.

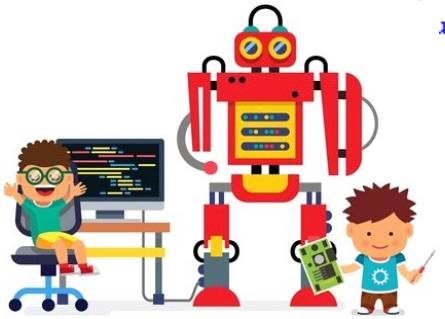
- En java se consigue por medio de **clases abstractas** e **interfaces**.

- Se posibilita que una clase que hereda, cambie sus atributos y métodos.




```java
public class TestEmpresa {
    public static class persona {
        private String nombre;
        public void setNombre(String nom){
          nombre = nom;
        }
        public String getNombre(){
          return nombre;
        }
    }
    public static class empleado extends persona
        protected int sueldoBase;
        public int getSueldo(){
            return sueldoBase;
        }
        public void setSueldoBase(int a){
            sueldoBase = a;
        }
    }
     ublic static class encargado extends empleado{
        public int getSueldo(){
            Double d = new Double(sueldoBase*1.1);
            return d.intValue();
        }
        public void setPuesto(String a){
            String puesto = a;
        }
```

Sobreescribe al anteri



```java
public static void main(String [] args){
    persona p1;
    p1 = new empleado();
    p1.setNombre("Jorge Durán");
    p1.setSueldoBase("100");
    empleado e1;
    e1 = new encargado();
    e1.setSueldoBase(500);
    e1.setPuesto("Jefe almacén");
    System.out.println(e1.getSueldo());
}
}
```

Referencia a persona

Llama a objeto empleado

```
run:
Sueldo: 550
BUILD SUCCESSFUL (total time: 2 seconds)
```

Lo solucionamos haciendo un cast.

```java
public static void main(String [] args){
    persona p1;
    p1 = new empleado();
    p1.setNombre("Jorge Durán");
    ((empleado)p1).setSueldoBase(100);
    empleado e1;
    e1 = new encargado();
    e1.setSueldoBase(500);
    ((encargado)e1).setPuesto("Jefe almacén");
    System.out.println("Sueldo: " + e1.getSueldo());
}
```

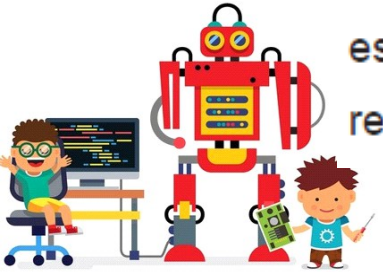
POO. UTILIZACIÓN AVANZADA DE CLASES

POLIMORFISMO.

En el código anterior creamos una referencia persona que apunta a un objeto de la clase empleado. La variable p1 podrá hacer llamadas a métodos de la clase persona pero no de la clase empleado, por lo tanto, la llamada al método setSueldoBase dará un error de compilación.

Por otra parte, vemos que el código anterior crea la referencia a empleado pero apunta a un objeto del tipo encargado. La llamada al método setPuesto dará error por lo explicado anteriormente pero no así la llamada al método getSueldo(). La pregunta que nos hacemos es la siguiente ¿El programa mostrará por pantalla 500 ó 550?

La solución es 550. Aunque la referencia se creo para la clase empleado y solamente se pueden llamar a métodos de dicha clase, el método getSueldo() está sobrescrito y como el apunta a un objeto de la clase encargado Java resuelve que tiene que ejecutar el método de dicha clase.



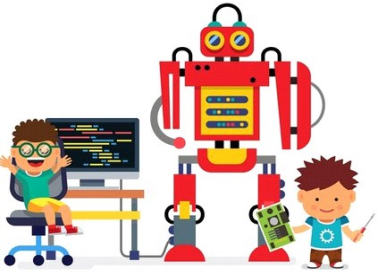
POO. UTILIZACIÓN AVANZADA DE CLASES

POLIMORFISMO.

Cuando la referencia se creó para la clase base, solamente se pueden hacer llamadas a métodos de dicha clase base.

En Java existen dos tipos de vinculaciones (con vinculación nos referimos a la llamada realizada a un método y el código que se va a ejecutar en dicha llamada), la vinculación temprana y la vinculación tardía.

- **Vinculación temprana.** Se realiza en **tiempo de compilación**. Con métodos normales o sobrecargados Java utiliza la vinculación temprana.
- **Vinculación tardía.** Se realiza en **tiempo de ejecución**. Cuando se redefinen métodos se realizará dicha vinculación (salvo métodos definidos como final).



POO. UTILIZACIÓN AVANZADA DE CLASES

POLIMORFISMO.

En nuestro caso se ha declarado el método getSueldo() en la clase empleado (clase padre) y se ha sobrescrito en la clase derivada encargado (clase hija). Cuando en tiempo de ejecución se llama a este método, el tipo de objeto al que apunta la variable prima sobre el tipo de la referencia. Es en tiempo de ejecución cuando se comprueba que aunque la referencia es de tipo empleado, la variable el apunta a un objeto de tipo encargado y el método de esta clase es el que se va a ejecutar.

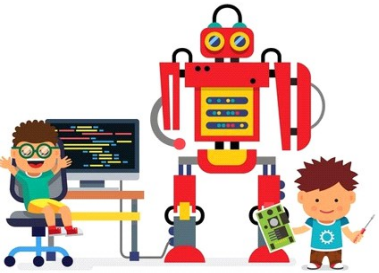
Aunque el polimorfismo confiere muchas ventajas para el lenguaje, hemos descubierto una limitación, y es el tipo de la referencia la que limita los métodos que podemos ejecutar (en nuestro ejemplo la llamada al método setPuesto dará error) o las variables miembro accesibles.

POO. UTILIZACIÓN AVANZADA DE CLASES

POLIMORFISMO.

Como hemos visto anteriormente nuestro programa va a dar errores de compilación. Entonces, ¿qué hacer para que el código anterior compile y funcione? La respuesta a esta pregunta es utilizar un cast explícito (obligar al compilador a transformar obligatoriamente el objeto en otro). El cast entre objetos se verá en profundidad en un apartado posterior y por lo tanto, de momento se incluyen las líneas que dan problema y debajo la línea reprogramada solventando dicho problema:

```
public static void main(String [] args){
    persona p1;
    p1 = new empleado();
    p1.setNombre("Jorge Durán");
    ((empleado)p1).setSueldoBase(100);
    empleado e1;
    e1 = new encargado();
    e1.setSueldoBase(500);
    ((encargado)e1).setPuesto("Jefe almacén");
    System.out.println("Sueldo: " + e1.getSueldo());
}
```



POO. UTILIZACIÓN AVANZADA DE CLASES

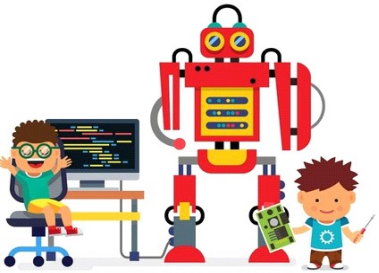
POLIMORFISMO.

Se puede crear un objeto cuya referencia sea una interfaz. Este objeto solamente podrá ejecutar los métodos de dicha interfaz pero no podrá utilizar los métodos y miembros de dicho objeto. La interfaz concebido así es una forma de unificación de uso entre clases muy diferentes.

```
public interface Dibujable{
     public void dibujar();
}
```

```
public class Rectangulo implements Dibujable{
     public void dibujar(){
          System.out.println("____________");
          System.out.println("|          |");
          System.out.println("|__________|");
     }
}
```

```
Dibujable grafico = new Rectangulo();
grafico.dibujar();
```



POO. UTILIZACIÓN AVANZADA DE CLASES

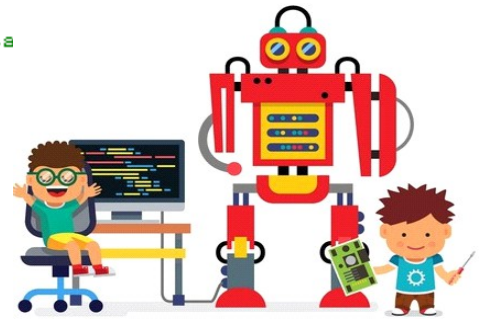
SOBREESCRITURA DE MÉTODOS (Overriding.)

- Permite modificar el comportamiento de la clase padre siempre que:
  - Tengan el mismo nombre.
  - El **return** sea del mismo tipo.
  - Deben conservar la misma lista de argumentos.

```java
public class AvesExoticas{
   public static class Pajaro {
      protected String nombre;
      protected String color;
      public String getDetalles(String nompaj, String colpaj){
          nombre = nompaj;
          color = colpaj;
         return "Nombre: " + nombre
                 + "\n Color: " + color;
      }
   }
   public static class Loro extends Pajaro {
      protected String pedigri;
      public String getDetalles(String nompaj, String colpaj){
         pedigri = "Pavo Real";
         nombre = nompaj;
         color = colpaj;
         return "Nombre: " + nombre
                 + "\n Color: " + color
                 + "\n Pedigrí: " + pedigri;
      }
   }
```

```java
public static void main(String [] args){
   Pajaro pio = new Pajaro();
   System.out.println(pio.getDetalles("Pichu","Verde"));
   Pajaro loro = new Loro();
   System.out.println(loro.getDetalles("Pavo","Azul"));
   Loro lorito = new Loro();
   System.out.println(lorito.getDetalles("Piopio","Naranja"));
}
```

```
run:
Nombre: Pichu
 Color: Verde
Nombre: Pavo
 Color: Azul
 Pedigrí: Pavo Real
Nombre: Piopio
 Color: Naranja
 Pedigrí: Pavo Real
BUILD SUCCESSFUL (tota
```

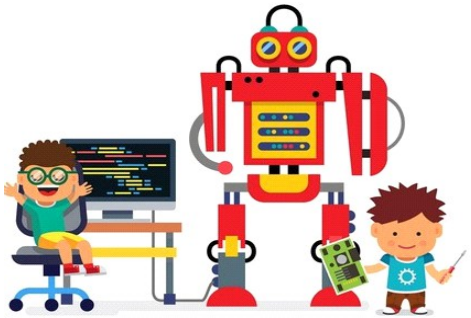


POO. UTILIZACIÓN AVANZADA DE CLASES

SOBREESCRITURA DE MÉTODOS (Overriding.)

Según el ejemplo anterior se puede observar lo siguiente:

- La clase Loro desciende de la clase Pájaro.
- La clase Loro sobrescribe el método getDetalles(), ambas con el mismo nombre.
- El método getDetalles() de clase padre e hija tienen la misma lista de argumentos.
- El método getDetalles() de clase padre e hija devuelven un objeto String (mismo tipo).
- El método getDetalles() de clase padre e hija tienen el mismo modificador de acceso (public).

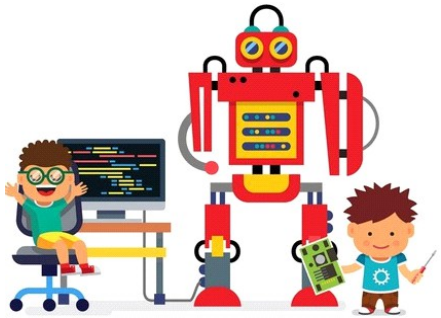


POO. UTILIZACIÓN AVANZADA DE CLASES

SOBRECARGA DE MÉTODOS (Overloading.)

La sobrecarga es la implementación varias veces del mismo método con ligeras diferencias adaptadas a las distintas necesidades de dicho método.

- Los métodos sobrecargados deben de cambiar la lista de argumentos obligatoriamente.
- Un método puede estar sobrecargado en la clase o en una subclase.
- Al sobrecargar un método se pueden utilizar las mismas excepciones o añadir algunas.
- Los métodos sobrecargados pueden cambiar el tipo de retorno o el modificador de acceso.

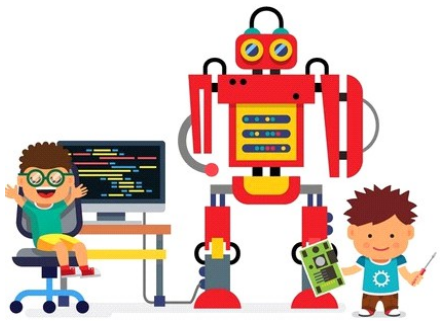


POO. UTILIZACIÓN AVANZADA DE CLASES

SOBRECARGA DE MÉTODOS (Overloading.)

```
public class AvesExoticas{
   public static class Pajaro {
      protected String nombre;
      protected String color;
      public String getDetalles(String nompaj, String colpaj){
          nombre = nompaj;
          color = colpaj;
         return "Nombre: " + nombre
                 + "\n Color: " + color;
      }
   }
   public static class Loro extends Pajaro {
      protected String pedigri;
      public String getDetalles(String nompaj, String colpaj, String pedipaj){
         pedigri = pedipaj;
         nombre = nompaj;
         color = colpaj;
         return "Nombre: " + nombre
                 + "\n Color: " + color
                 + "\n Pedigrí: " + pedigri;
      }
   }
   public static void main(String [] args){
      Pajaro pio = new Pajaro();
      System.out.println(pio.getDetalles("Pichu","Verde"));
      Pajaro loro = new Loro();
      System.out.println(loro.getDetalles("Pavo","Azul"));
      Loro lorito = new Loro();
      System.out.println(lorito.getDetalles("Piopio","Naranja","Pavo Real"));
   }
}
```

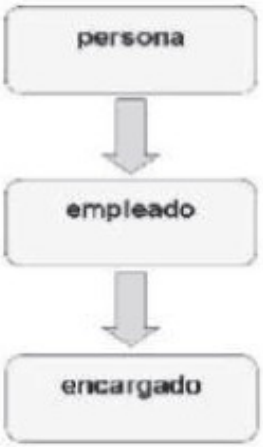
```
run:
Nombre: Pichu
 Color: Verde
Nombre: Pavo
 Color: Azul
Nombre: Piopio
 Color: Naranja
 Pedigrí: Pavo Real
BUILD SUCCESSFUL (total time: 3 seconds)
```



POO. UTILIZACIÓN AVANZADA DE CLASES

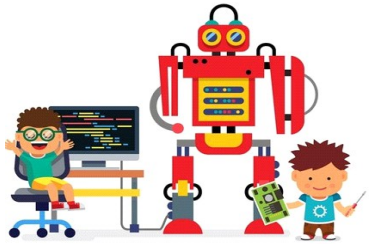
CONVERSIONES ENTRE OBJETOS (CASTING)

Imaginemos el siguiente árbol de jerarquía:



Si por ejemplo tenemos un método m() que espera un argumento tipo empleado, podemos pasarle un objeto de tipo persona **(1)** o un objeto de tipo encargado **(2)**.

Opción 1. Si le pasamos un objeto de tipo persona nos encontramos con una pérdida de precisión puesto que no se pueden ejecutar todos los métodos de los que dispone un objeto de tipo empleado. Recuerda que persona contiene menos métodos que la clase empleado.



En este caso es necesario hacer un casting, sino el compilador dará error.

POO. UTILIZACIÓN AVANZADA DE CLASES

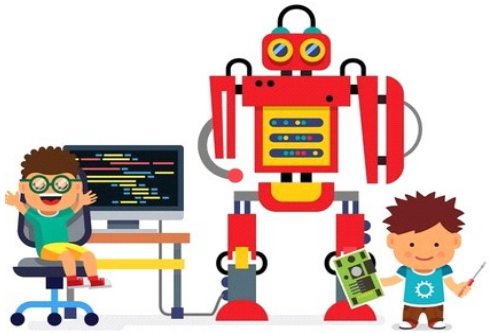
CONVERSIONES ENTRE OBJETOS (CASTING)

```
public static void main(String [] args){
    persona p1 = new empleado();
    p1.setNombre("Jorge Durán");
    m(p1);//da error, puesto que persona es clase padre.
    m((empleado)p1);//se necesita casting.
    encargado en1 = new encargado();
    en1.setNombre("Isaac Asimov");
    m(en1);//no se hace casting, encargado hereda de empleado
    persona p2 = new persona();
    p2.setNombre("Juan Serrano");
    m((empleado)p2);//Da error en ejecución
                    //no podemos hacer casting ya la instancia p2
                    //apunta a un objeto de tipo persona.
}
```

Como se puede observar tenemos un método m() que espera como parámetro un objeto de tipo empleado. En primer lugar haremos que la referencia p1 a persona apunte al objeto de tipo empleado(). Una vez hecho esto no tendremos problemas para utilizar esta referencia en el método. Para ejecutar el método tendremos que hacer **explícitamente un casting** como se puede ver en la siguiente línea de código:

```
m ((empleado)p1);
```

Este casting es similar a lo que hemos visto antes con los métodos primitivos.



POO. UTILIZACIÓN AVANZADA DE CLASES

CONVERSIONES ENTRE OBJETOS (CASTING)

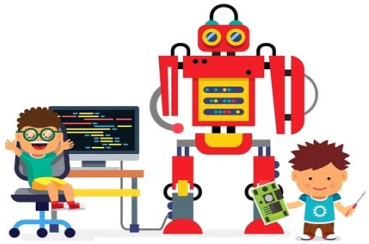
Si la referencia p1 en vez de apuntar a un objeto de tipo empleado apuntase a un objeto de tipo persona. En ese caso utilizando siguiente línea de código:

```
persona p1 = new persona();
```

Java dará un error en ejecución. Java lanzará una excepción del tipo ClassCastException y avisará que no es posible realizar el casting "persona cannot be cast to empleado".

Opción 2. Pasamos un objeto de tipo encargado. En este caso al ser una subclase no tendremos problemas.

Los errores se producen cuando se llama a métodos que el objeto destino no tiene.



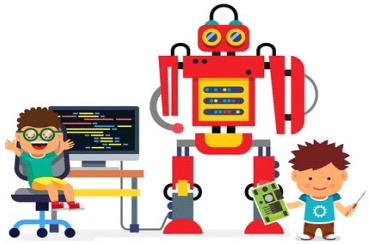
POO. UTILIZACIÓN AVANZADA DE CLASES

CONVERSIONES ENTRE OBJETOS (CASTING)

```
empleado emp = new empleado();
encargado enc = new encargado();
emp = enc; //No necesita casting
enc = (encargado)emp; //necesita casting explícito
```

Las reglas seguidas en el ejemplo anterior son las siguientes:

- Cuando se utiliza una clase más específica (más abajo en la jerarquía) no hace falta casting.
- Cuando se utiliza una clase menos específica (más arriba en la jerarquía) hay que hacer un casting explícitamente.



POO. UTILIZACIÓN AVANZADA DE CLASES

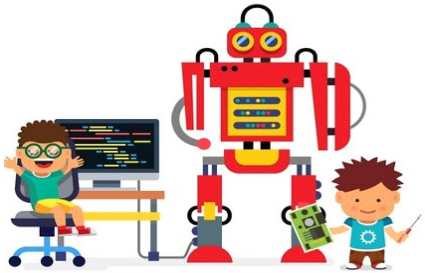
ACCESO A MÉTODOS DE LA SUPERCLASE

Para acceder a los métodos de la superclase se utilizará la palabra reservada **super**. Esta palabra reservada está disponible en cualquier método no estático de una subclase.

Es importante tener en cuenta que super es una referencia al objeto actual teniendo en cuenta la instancia de su superclase.

```
public class padreSuper {
  public static class padre{
      protected int dato;
      public void m(){
          System.out.println("Método clase padre");
      }
  }
  public static class hijo extends padre{
     private int dato;
     public void m(){
         System.out.println("Método clase hijo");
         super.dato = 10;
         dato = 20;
     }
     public void getDato(){
         System.out.println(super.dato);
     }
     public void mostrar(){
         this.m();
         m();
         super.m();
     }
  }
  public static void main(String [] args){
     hijo h = new hijo();
     h.mostrar();
     h.getDato();
  }
}
```

```
run:
Método clase hijo
Método clase hijo
Método clase padre
10
BUILD SUCCESSFUL (total time: 0 seconds)
```



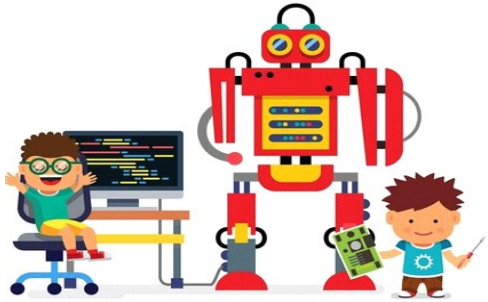
POO. UTILIZACIÓN AVANZADA DE CLASES

ACCESO A MÉTODOS DE LA SUPERCLASE

- Para acceder a métodos sobrescritos de la superclase utilizamos la palabra super.
- Es posible acceder a miembros protected de la superclase utilizando la palabra super.

this se utiliza para acceder a campos y métodos de la clase y super para la superclase. No importa que estos estén sobrescritos.

Cambiemos un poco el código anterior y veamos cómo lo visto anteriormente con super y this es aplicable también en las llamadas a los constructores de los objetos.



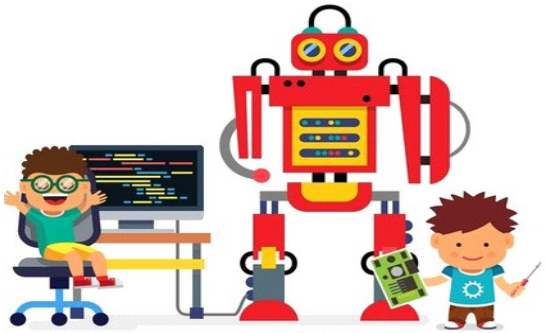
POO. UTILIZACIÓN AVANZADA DE CLASES

ACCESO A MÉTODOS DE LA SUPERCLASE

Obsérvese como se utiliza this y super en las llamadas a los constructores.

```
public class padreSuperConst {
        public static class padre{
        protected int dato1, dato2;
        padre(int x, int y){dato1=x;dato2=y;}
        padre(){this(5,5);}
    }
    public static class hijo extends padre{
        private int dato1, dato2;
        hijo(int x, int y){
            super(2,2);
            dato1 = x;
            dato2 = y;
        }
        hijo(){
            dato1 = 3;
            dato2 = 3;
        }
        public void getDato(){
            System.out.println("Padre Dato1: "+super.dato1);
            System.out.println("Padre Dato2; "+super.dato2);
            System.out.println("Hijo Dato1: "+this.dato1);
            System.out.println("Hijo Dato2: "+this.dato2);
        }
    }
    public static void main(String [] args){
        hijo h1 = new hijo(1,1);
        h1.getDato();
        hijo h2 = new hijo();
        h2.getDato();
    }
}
```

```
run:
Padre Dato1: 2
Padre Dato2; 2
Hijo Dato1: 1
Hijo Dato2: 1
Padre Dato1: 5
Padre Dato2; 5
Hijo Dato1: 3
Hijo Dato2: 3
BUILD SUCCESSFUL (total time: 0 seconds)
```



POO. UTILIZACIÓN AVANZADA DE CLASES

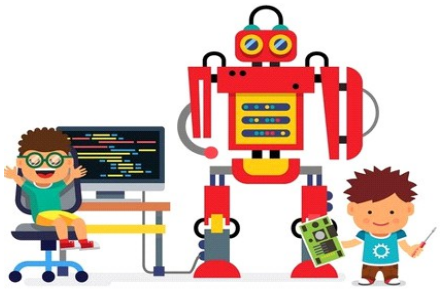
CLASES ANIDADAS

- Las clases anidadas tienen acceso a todos los métodos y atributos de la clase externa.
- Pueden ser private, public, protected o privada al paquete.
- Clases anidadas estáticas → static.
- Clases anidadas no estáticas → clases internas. No static

  -externa.interna objetointerno = objetoexterno.new interna();

USO:

- Si sólo se van a utilizar en un único lugar → hará el código más legible.
- Se incrementa la encapsulación dado que la clase anidada sólo se necesita en la clase externa.



```
public class Externa {

    public class Interna {
        public void imprimir() {
            System.out.println("Clase interna");
        }
    }

    public void imprimir() {
        System.out.println("Clase externa");
    }
}
```

```
public class PruebaClaseInterna {

    public static void main(String[] args) {
        Externa externa=new Externa();
        Externa.Interna interna=externa.new Interna();
        interna.imprimir();
        externa.imprimir();
    }

}
```

TEMA 5: PROGRAMACIÓN ORIENTADA A OBJETOS. UTILIZACIÓN AVANZADA DE CLASES